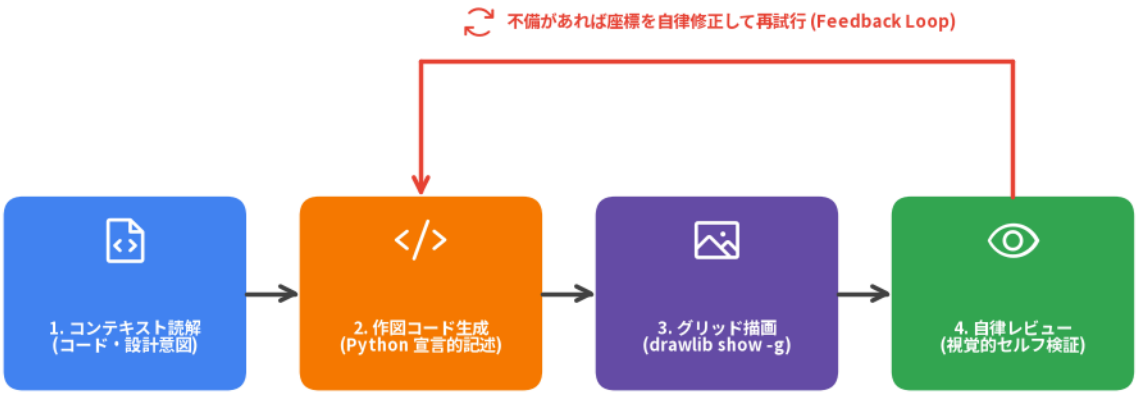
AIとDrawlibによるドキュメント自動生成

AI時代の "Illustrated Documentation as Code"



AI Agent による作図・視覚的自律検証ループ

作成者: Drawlib Core Team
対象: ソフトウェアエンジニア・アーキテクト・AI ペアプログラマー
バージョン: 0.3.0
発行日: 2026年10月

Table of Contents

第1章: AI 開発における作図・ドキュメントの課題

第2章: 解決アプローチ: Illustrated Documentation as Code

第3章: Drawlib の全体像と主要機能

第4章: インストールと環境構築

第5章: プロジェクト作成とビルド運用

第6章: AI エージェントとの連携・インストラクション

第7章: ベーシックな描画（Primitives）

第8章: SmartArts（構造化ビジュアル）

第9章: チャート描画（定量データの可視化）

第10章: システムダイアグラム（アーキテクチャ・設計図）

第11章: スタイルとテーマのカスタマイズ

第12章: プレゼンテーションスライドの作成 (`slide`)

第1章: AI 開発における作図・ドキュメントの課題

AI によるコード生成やペアプログラミングが日常的になる中、ソフトウェア開発におけるボトルネックは「コードの実装」から「**設計・アーキテクチャの可視化とドキュメントの同期**」へとシフトしています。

1.1 設計書と実装の乖離（ドキュメントの腐敗）

AI ツール（Cursor, Claude, Copilot など）を活用することで、プログラムコードは驚異的なスピードでリファクタリングや機能追加が行われます。しかし、システム全体のアーキテクチャ図やシーケンス図、データベーススキーマ図などのドキュメントは更新されずに置き去りにされがちです。

結果として、以下のような問題が発生します:

- **ドキュメントの形骸化**: 「ドキュメントと現行コードのどちらが正しいのかわからない」状態に陥る。
- **オンボーディングの遅延**: 新規メンバーや別の AI セッションが全体構造を正しく把握できない。
- **暗黙知の増大**: 最新の設計意図がコードのコミットログやチャット履歴にしか残らない。

1.2 手動 GUI ツールの限界

従来、エンジニアは以下のような GUI ツールを用いて作図を行ってきました:

- draw.io / diagrams.net
- Figma / Miro
- PowerPoint / Google Slides

これらのツールで作成された図版は通常、**PNG や JPEG などのバイナリ画像**としてリポジトリに保存されます。このバイナリ管理には決定的な欠点があります:

1. **Git で差分（diff）が追えない**: どこが変更されたのかプルリクエストで視覚的に追跡できません。
2. **AI Agent が編集できない**: AI はバイナリ PNG の中身を直接編集することができません。図を直すには人間が手動でツールを開いて修正し、再度エクスポートする手間が発生します。
3. **デザインの不統一**: 作成者ごとにフォント、色、線の太さ、余白がバラバラになり、組織としてのドキュメント品質が保てません。

1.3 AI に生の SVG や Matplotlib を書かせると破綻する理由

では、AI にコードで図を描かせれば解決するでしょうか？ 多くの開発者が試みて挫折するのが、**生 SVG** や **Matplotlib** の直接生成です:

- **生 SVG の直接生成**:
  - テキストの折り返し計算やフォント幅の算出が難しく、テキストが枠からはみ出す。
  - 矢印やコネクタの座標計算が複雑で、線の交差や位置ずれが多発する。
  - 何百行もの `<path>` や `<rect>` が生成され、人間が保守・レビュー不能になる。
- **Matplotlib の流用**:
  - 本来は学術的な散布図や折れ線グラフのためのライブラリであり、システム構成図やシーケンス図を描くための高レベルなプリミティブ（ノード、グループ、矢印配線、アイコン）が存在しない。
  - 大量のボイラープレートコードが必要となり、AI も構文エラーを起こしやすい。

こうした背景から生まれたのが、Python による **"Illustration as Code"（コードとしての図版）** を実現するライブラリ **Drawlib** です。

第2章: 解決アプローチ: Illustrated Documentation as Code

Drawlib は、ソフトウェアエンジニアと AI コーディングエージェントのための **"Illustrated Documentation as Code" (IDaC)** を実現する Python ライブラリです。従来の Documentation as Code（DaC）がテキスト中心であったのに対し、Drawlib は「図解・設計図がファーストクラスで統合された技術文書」をコードだけで構築します。

2.1 "Illustrated Documentation as Code" のコア思想

Drawlib のアプローチはシンプルです:

1. **宣言的 Python スクリプト**: 直感的な高レベル API で図版を定義します。
2. **Markdown への直接インライン埋め込み**: ドキュメントの中に `drawlib` ブロックとして作図コードを直接記述します。
3. **完全なバージョン管理**: 図版の定義コードが Markdown テキストとして Git で管理され、プルリクエストでコード差分としてレビュー可能です。
4. **自動コンパイル**: CLI ビルド（`drawlib build`）によって、静的 HTML サイト、GitHub 向け Markdown、高品質な PDF レポートへと自動レンダリングされます。

2.2 従来のアプローチとの比較

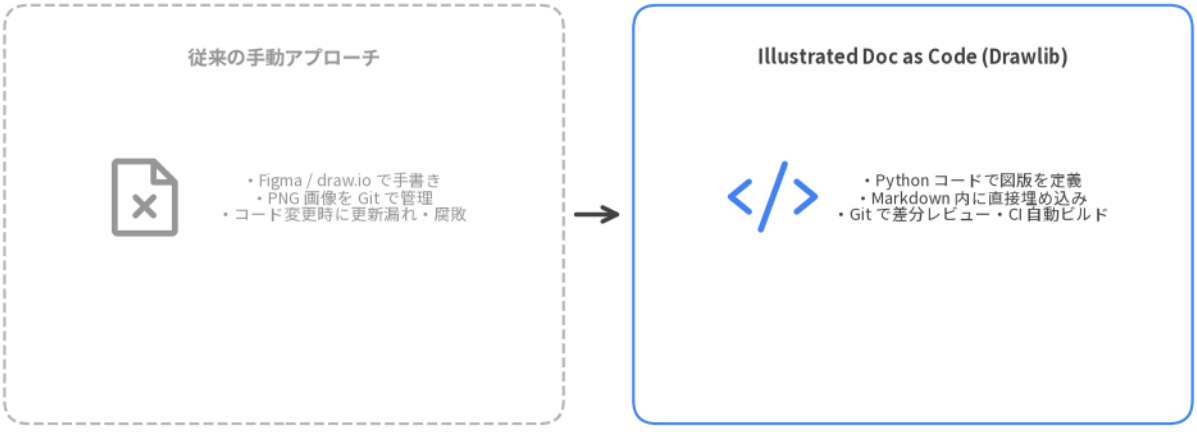


図 2.1: 従来の手動管理と Drawlib アプローチの比較

2.3 AI ペアプログラミングとの相乗効果

コードとして表現できることで、AI Agent（Cursor, Claude, Gemini 等）との協調作業が飛躍的に容易になります:

- **AI が仕様から図を自動生成**: 新しい API エンドポイントやデータモデルを実装した際、AI に「アーキテクチャ図を更新して」と指示するだけで図版コードが生成されます。
- **マルチモーダル検証**: AI Agent は自身が生成した図版を画像としてレンダリングし、配置やテキストの重なりを視覚的にセルフレビューして修正できます。
- **再現性（100% Deterministic）**: 同じコードからは常にピクセル単位で同一の図版が再現されます。

第3章: Drawlib の全体像と主要機能

Drawlib は、"Illustrated Documentation as Code" を支える統合描画・ドキュメント作成プラットフォームです。ドキュメント構築パイプラインに加え、低レベルの基本図形描画から高度なシステムアーキテクチャ設計までをカバーする 4 つの描画レイヤーを提供しています。

3.1 描画・ドキュメントコンポーネントレイヤー

レイヤー	提供モジュール	主な用途・描画対象
1. Primitives (基本要素)	`drawlib.shapes`, `lines`, `text`, `icons`, `images`	23種の幾何図形、折れ線・曲線矢印、リッチテキスト、Phosphor/FontAwesome/GCP アイコン
2. SmartArts (構造化)	`drawlib.smartarts`	プロセスシェブロン、表組み、マインドマップ、ツリー階層、サイクル図、吹き出し
3. Charts (定量データ)	`drawlib.charts`	ガントチャート、棒グラフ、折れ線グラフ、面グラフ、円グラフ、レーダー、散布図
4. Diagrams & Graphs (システム設計)	`drawlib.diagrams`, `drawlib.graph`	クラウド構成図、フロー図、シーケンス図、ER図、クラス図、ステート図、Pure Python 自動レイアウトソルバー
5. Presentation & Animation	`drawlib.slide`, `drawlib.anim`	16:9 プレゼンテーションスライドデッキ、APNG / Animated WebP ネイティブ生成
6. Builder & CLI (ドキュメント基盤)	`drawlib._builder`, `drawlib._cli`	Markdown からの HTML / PDF / Markdown / 画像一括コンパイル、プレビューサーバー

3.2 統一デザインシステムとセマンティックカラー

Drawlib では、作図のたびに RGB 値を迷う必要はありません。洗練された **セマンティック 6 色パレット** とスタイル命名規則が標準で組み込まれています。

- `Styles.Primary`: システムの中核ノードや主動線（青系）
- `Styles.Secondary`: 外部システムやクライアント（スレート・インディゴ系）
- `Styles.Accent`: 注目させたいハイライトや補助サービス（アンバー・ティール系）
- `Styles.Muted`: 境界枠や補助注記、背景コンテナ（薄いグレー系）
- `Styles.Success` / `Styles.Danger`: 正常ステータス（緑系）および異常・警告・エラー（赤系）

各カラーには `PrimaryFlat`（塗りつぶし）、`PrimaryOutline`（枠線）、`PrimaryBold`（太線）、`PrimaryDashed`（破線）などの PascalCase 定数トークンが用意されています。

テキストおよび接続線のニュートラルカラー原則

視認性の確保と見やすいレイアウトのため、以下の基本ルールが定められています:

- **ニュートラルタイポグラフィ**: 背景が明るい場合は `Styles.Dark` または `Styles.DarkBold` を基本とします。背景が濃い塗りつぶしノード内部では `Styles.WhiteBold` または `Styles.Light` を使用します（意味なく文字に `Primary` や `Secondary` を選ぶことは避けます）。
- **ニュートラルコネクタ**: 通常の処理進行や接続線・矢印は `Styles.DarkBold` を推奨します。`Styles.Primary` や `Styles.Danger` などのカラー矢印は、クリティカル動線やエラーロールバックなど、強調すべき意味がある場合のみ使用します。

3.3 日本語フォント・CJK の完全サポート

Drawlib は、日本語を含む多言語タイポグラフィをファーストクラスでサポートしています。 `--lang ja` を指定してプロジェクトを作成すると、以下の環境が自動的に整います:

- **HTML / PDF 出力**: Google Fonts（Noto Sans JP）およびローカルシステムフォント（Hiragino Sans, Yu Gothic）の適切なフォントスタックが適用されます。
- **キャンバス描画**: `drawlib.fonts.FontJapanese` が自動適用され、図形内の日本語テキストの文字化けやレイアウト崩れを防ぎます。

第4章: インストールと環境構築

Drawlib は、モダンな Python パッケージマネージャー `uv` を用いて、数秒でセットアップできます。

4.1 前提環境

- **Python**: 3.11 以上
- **パッケージマネージャー**: `uv`（推奨）または `pip`

4.2 Git からのワンライナー導入

PDF 出力機能（`playwright` 連携）を含めた完全な環境を Git リポジトリから直接インストールします:

```
# 1. PDF エクストラ付きで drawlib をインストール
uv add "drawlib[pdf] @ git+https://github.com/yuichi110/drawlib.git"

# 2. PDF 生成に必要な headless Chromium ブラウザをダウンロード
uv run playwright install chromium
```

> **Note (Linux ヘッドレスサーバー環境)**:
> Ubuntu 等のサーバー環境で OS 共有ライブラリが不足している場合は、以下のように `--with-deps` を付与して実行してください:
> `uv run playwright install --with-deps chromium`

4.3 インストールの動作確認

インストールが完了したら、以下のコマンドで CLI が利用可能であることを確認します:

```
# バージョンの確認
uv run drawlib --version

# 利用可能なプロジェクトテンプレート一覧の表示
uv run drawlib init list
```

以下のように `site`, `simple`, `pdf`, `image` が表示されれば環境構築は完了です。

第5章: プロジェクト作成とビルド運用

Drawlib には、目的別のドキュメントプロジェクトを瞬時にスキャフォールドする `drawlib init` コマンドが用意されています。

5.1 プロジェクトテンプレートの種類

テンプレート	用途	主な生成成果物
`doc`	複数章の技術仕様書、設計レポート、提出資料	`<target>.html`, `<target>.pdf`, `<target>.md`
`site`	サイドバーナビゲーション付きの多ページドキュメントサイト	`<target>_html/` および `<target>/`
`slide`	16:9 プレゼンテーションスライド（Web 発表・PDF 配布）	`<target>/index.html` および `<target>.pdf`
`images`	独立した Python 作図スクリプト集	`<target>/*.png`, `*.webp`

5.2 日本語 Google スタイルのドキュメントプロジェクト作成

以下のコマンド 1 つで、日本語対応および Google スタイルの線形ドキュメントプロジェクト（`report_src/`）を作成できます:

```
uv run drawlib init doc report -l ja -s google
```



図 5.1: プロジェクト作成から成果物生成までのライフサイクル

ターゲット引数による名前の連動仕様

`drawlib init doc report` を実行すると、以下の連動関係が自動的に構築されます:

- **ソースフォルダ**: `report_src/` がカレントディレクトリに作成されます。
- **ビルド出力**: `report.pdf`, `report_html/`, `report_markdown/` が出力対象として自動設定されます。

5.3 ディレクトリ構造

生成された `report_src/` 内の各ファイルは以下の役割を持ちます:

```
report_src/
├── 00_cover.md           # [必須] 表紙（タイトル・メタデータ）
├── 01_overview.md        # 第1章ドキュメント
├── 02_design.md          # 第2章ドキュメント
├── style.css             # PDF デザイン用スタイルシート (Google テーマ)
├── styles.py             # Python 作図用スタイル設定 (GoogleStyles + 日本語)
├── template.html         # Jinja2 印刷レイアウトテンプレート
├── utils.py              # プロジェクト共通の作図ヘルパー関数
├── build.sh              # [実行可能] ワンクリックビルドスクリプト
└── README.md             # プロジェクト手引書
```

5.4 ビルドの実行

ドキュメントを PDF や HTML にコンパイルするには、生成された `build.sh` を実行するだけです:

```
# ビルドスクリプトを実行
./report_src/build.sh
```

ビルドが完了すると、`report.pdf` および関連成果物が出力されます。

第6章: AI エージェントとの連携・インストラクション

Drawlib は、AI コーディングアシスタント（Claude, Cursor, Gemini, Copilot）が自律的かつ正確に作図を行えるよう「Agent-First」で設計されています。エージェントに基本 instruction を登録した後は、AI が Drawlib CLI と自律的に対話して必要な機能を調査し、検証済みの図解ドキュメントを出力します。



図 6.1: Drawlib と AI Agent の自律対話・ドキュメント出力モデル

6.1 組み込みルールシステムの活用 (`drawlib rules`)

Drawlib には、ライブラリ仕様や作図ベストプラクティスが最初からルールとして組み込まれています。ターミナルからオンデマンドでエージェントに知識を提供できます:

```
# 利用可能なルール一覧
uv run drawlib rules list

# エージェント用指示マニュアル
uv run drawlib rules show agent-instruction

# キャンバス座標系と描画ライフサイクル
uv run drawlib rules show overview

# 各モジュールの詳細仕様
uv run drawlib rules show lib-smartarts
uv run drawlib rules show lib-diagrams
uv run drawlib rules show lib-charts
```

6.2 プロジェクトルールファイルへの設定

Cursor（`.cursorrules`）や Claude Code / Gemini（`.agents/rules/`）に以下の基本ルールを登録しておくと、AI が自動的に Drawlib の作図原則に従うようになります:

```
# Drawlib 作図ルール
- 原則として生 SVG や低レベル matplotlib は書かず、drawlib の高レベルモジュール（smartarts, diagrams, charts）を使用すること。
- キャンバスの原点（0, 0）は「左下」である。
- スタイルとカラーは常に PascalCase（Styles, Colors）でインポート・参照すること。
- Markdown 内のコードブロックには必ず `file:<名前>.png` と `caption:"..."` を指定すること。
```

6.3 エージェントの自律フィードバックループ

AI エージェントが図版を作成する際は、以下のセルフレビュー手順を踏ませることで、位置ズレや重なりのない完璧な図版が得られます:

1. **コードの記述**: Markdown 内に `drawlib` ブロックを記述。
2. **グリッド付きプレビュー出力**:

   ```
   uv run drawlib show doc.md arch.png -g -o .drawlib/scratch/arch.png
   ```

3. **画像ツールの確認**: エージェントが画像ファイルを視覚的に確認し、テキストのはみ出しや余白の不均等さを検査。
4. **座標の微調整**: 必要に応じて座標を修正し、再確認。

第7章: ベーシックな描画（Primitives）

Drawlib の基本描画レイヤー（`shapes`, `lines`, `text`, `icons`）を使用すると、任意の幾何学図形や自由なレイアウトを自在に構築できます。

7.1 キャンバスの座標系

Drawlib は、数学的な **直交座標系（デカルト座標系）** を採用しています:

- **原点** `(0, 0)`: キャンバスの **左下** です（画面描画系の左上原点とは異なります）。
- **サイズ指定**: `setup(width=W, height=H)` でキャンバスサイズを定義します。

7.2 基本図形（Shapes）と矢印（Lines）

22 種類の図形プリミティブ（四角形、円、楕円、扇形、ドーナツ、矢印ポリゴン、星型など）が用意されています。

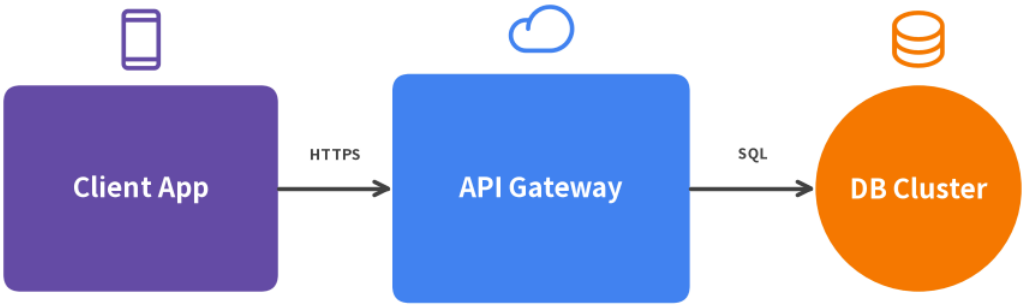


図 7.1: 基本図形・矢印・アイコンを組み合わせたサービス連携図

7.3 主要な描画関数一覧

- `rectangle(xy, width, height, style=...)`: 長方形（`style.shape_r` で角丸対応）
- `circle(xy, radius, style=...)`: 円
- `line(start_xy, end_xy, arrow_head="->", style=...)`: 矢印付き直線
- `line_curved(start_xy, end_xy, bend=0.2, ...)`: 滑らかな曲線
- `text(xy, text="...", style=...)`: スタイル付きテキスト
- `phosphor.<icon_name>(xy, width, style=...)`: Phosphor ベクターアイコン

第8章: SmartArts（構造化ビジュアル）

`drawlib.smartarts` は、手作業で個々の四角形や直線の座標を計算することなく、洗練された構造化図版を自動レイアウトする高レベルモジュールです。

8.1 利用可能な SmartArts コンポーネント

- **`ChevronProcess`**: 矢羽型（シェブロン）による工程・デプロイパイプライン表現。
- **`Table`**: 均等セル配置、ヘッダー強調、罫線装飾を備えたデータ比較テーブル。
- **`MindMapNode`**: 放射状または特定方向に広がるアイデアマップ・機能構造図。
- **`TreeNode`**: 組織図やディレクトリツリーを表す階層構造図。
- **`Cycle`**: フィードバックループや継続的改善サイクル図。
- **`Pyramid`**: テストピラミッドやセキュリティ階層モデル。
- **`BoxList`** / **`BulletPoints`**: 箇条書きやカードスタックの整列配置。
- **`GridLayout`**: 均等グリッドによるダッシュボード風レイアウト。
- **`SourceCode`**: シンタックスハイライト付きコードブロック表示。

> [!NOTE] 特定のノードを指し示すコールアウト吹き出しは、基本図形モジュール `drawlib.shapes.bubblespeech` から利用できます。

8.2 パイプラインと機能比較の作成例



カテゴリ	主要コンポーネント	メリット
SmartArts	ChevronProcess, Table, MindM	手作業の座標計算が不要
Diagrams	Architecture, Sequence, Flo	業界標準記法に準拠した設計図
Charts	Gantt, Bar, Line, Pie	ロードマップや進捗の可視化

図 8.1: ChevronProcess と Table を組み合わせたリリースフロー

第9章: チャート描画（定量データの可視化）

`drawlib.charts` は、プロジェクトのロードマップ、スケジュール管理、パフォーマンスメトリクスを美麗にレンダリングするチャートモジュールです。

9.1 ガントチャート (`GanttChart`)

開発スケジュールやマイルストーン、タスク間の依存関係（先行タスクと後続タスクの矢印）を視覚化できます。

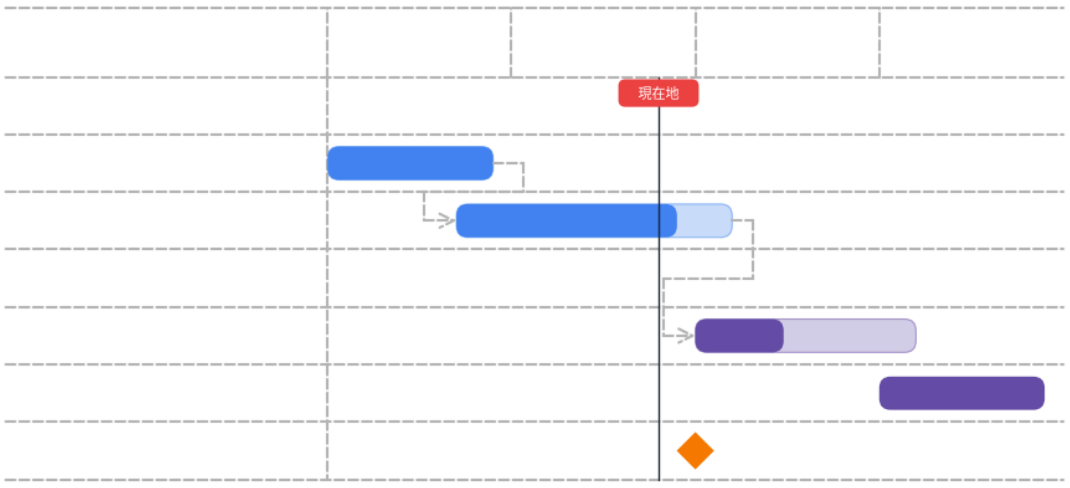


図 9.1: エンジニアリング開発ロードマップ

9.2 その他のチャート

- `BarChart`: 垂直・水平棒グラフ、グループ化・積み上げ対応。
- `LineChart` / `AreaChart`: 時系列推移、多系列プロット。
- `PieChart`: 構成比率（4% 未満の微小スライスは自動でテキスト重なり防止）。
- `RadarChart`: 多変量評価、スキルマトリクス。
- `ScatterChart`: 散布図、データ分布・レイテンシ相関の可視化。

第10章: システムダイアグラム（アーキテクチャ・設計図）

`drawlib.diagrams` は、複雑なソフトウェアシステムやネットワーク構造をエンジニアリング標準記法で描画するための最上位モジュールです。

10.1 アーキテクチャ図 (`ArchitectureDiagram`)

VPC やサブネットなどの境界グループ（`NodeGroup`）、コンポーネント（`Node`）、サービス間の直交配線を直感的に定義できます。

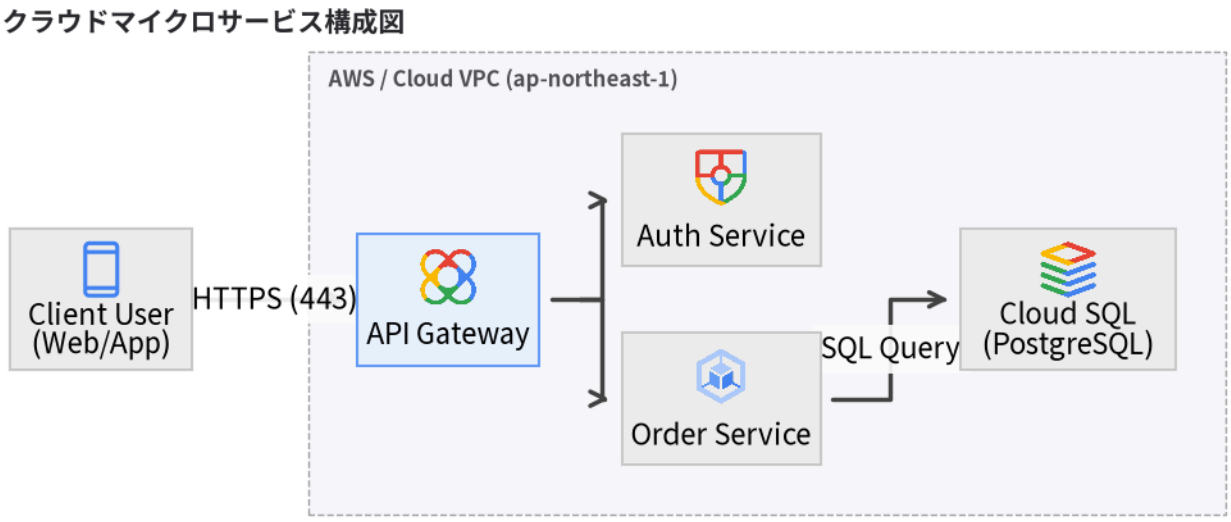


図 10.1: クラウドマイクロサービス構成図

10.2 自動レイアウトエンジン (`drawlib.graph`)

ゼロからノードのトポロジー（論理関係）とピクセル座標（幾何美観）を同時に計算しようとすると、人間にとっても AI にとってもワーキングメモリが逼迫します（**コールドスタート問題**）。

Drawlib は Pure Python の自動レイアウトソルバー `drawlib.graph` を提供しており、外部ツール（Graphviz 等）に依存することなく、宣言的な記述から美しい座標を自動算出できます。

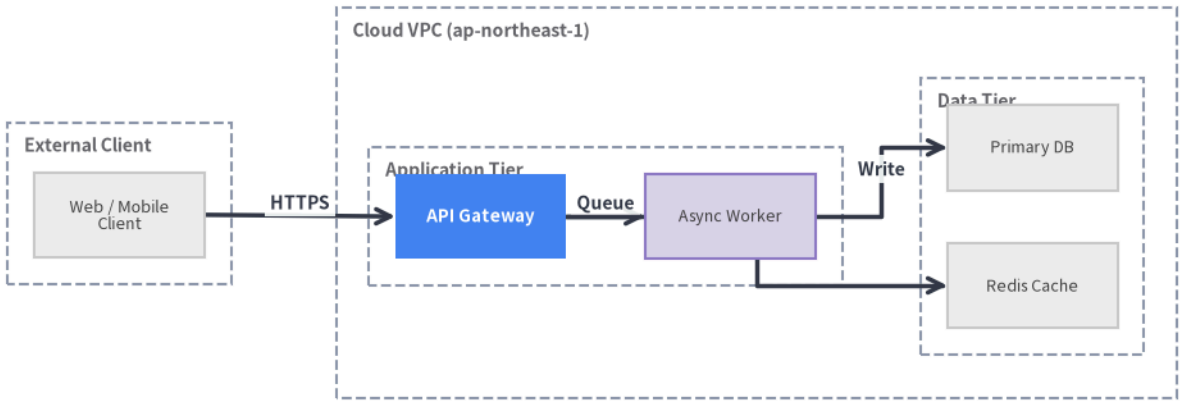


図 10.2: ArchitectureGraph による自動レイアウトとクラスタリング

下書きから清書への架け橋 (`export_code()`)

- `g.draw()` : 宣言したトポロジーを自動レイアウトして即座にキャンバスに描画します。
- `g.export_code()` : 計算された座標を変数化（例: `api_xy = (72.0, 42.0)`）した Drawlib Python コードを自動生成します。「**大枠は自動配置で下書きし、細部だけ手動で絶対座標微調整を行う**」という理想的な 2 段階ワークフローが完結します。

専門用途に応じた 5 大ソルバー

- **`ArchitectureGraph`** : 多層階層（Tier）と VPC / サブネット境界、方位（Compass: left/right/top/bottom/center）配置。
- **`LayerGraph`** : Sugiyama 法に基づく多段 DAG・データフロー・依存関係グラフ。
- **`TreeGraph`** : Reingold-Tilford 法による組織図・決定木・AST。
- **`RadialGraph`** : 同心円（Concentric Rings）によるイベント駆動・ハブ＆スポーク。
- **`GridGraph`** : 2D 行列グリッド整列とサービス一覧。

10.3 その他の専門ダイアグラム

- **`FlowDiagram`** : 処理ステップ、条件分岐（Decision）、ループ制御。
- **`SequenceDiagram`** : アクター、参加者、同期/非同期メッセージ、ノート。
- **`ERDiagram`** : 物理テーブル定義、PK/FK、Crow's Foot（鳥の足）記法によるリレーション。
- **`ClassDiagram`** : オブジェクト指向クラス、メソッド、継承・集約関係。
- **`StateDiagram`** : 状態遷移マシン、イベントトリガー、ガード条件。

第11章: スタイルとテーマのカスタマイズ

Drawlib では、ドキュメント全体の配色や作図トーンをプロジェクト単位で柔軟にカスタマイズできます。

11.1 `styles.py` による作図テーマの制御

プロジェクトのルート（またはソースディレクトリ）にある `styles.py` で、作図時に使用される `Colors` と `Styles` を決定します:

```
from drawlib.fonts import FontJapanese
from drawlib.preset_colors import GoogleColors
from drawlib.preset_styles import GoogleStyles

# カラーパレットとスタイルの初期化
Colors = GoogleColors()
Styles = GoogleStyles().patch_font(
    regular=FontJapanese.SANSSERIF_REGULAR,
    bold=FontJapanese.SANSSERIF_BOLD,
    thin=FontJapanese.SANSSERIF_THIN,
)
```

11.2 利用可能なスタイルプリセット

- **`DefaultStyles`** / **`DefaultColors`**: Tailwind / VitePress にインスパイアされた開発者向け標準テーマ。
- **`GoogleStyles`** / **`GoogleColors`**: Material Design および Google Docs/Slides に準拠した親しみやすいテーマ。
- **`MonochromeStyles`** / **`MonochromeColors`**: 印刷物や公的文書に適した白黒・グレースケールテーマ。

11.3 既存スタイルの部分変更 (`.patch()`)

既存のスタイルをベースに、特定の色や線幅、フォントサイズだけを変更した新しいスタイルを簡単に派生できます:

```
from drawlib.styles import Styles

# PrimaryFlat をベースに角丸と背景色を変更
my_card_style = Styles.PrimaryFlat.patch(
    shape_fill_color=(235, 248, 255),
    shape_line_color=(49, 130, 206),
    shape_line_width=1.5,
)
```

11.4 ドキュメントスタイル (`style.css`) との連動

`drawlib init` で `--style google` を指定すると、`styles.py` だけでなく `style.css` にも Google テーマ（青のアクセントカラー、Google Sans フォントスタック、クリーンなカード余白）が展開されます。これにより、**本文のテキストと作図のビジュアルが完全に調和した美しいドキュメント** が自動的に仕上がります。

第12章: プレゼンテーションスライドの作成 (`slide`)

Drawlib は、技術文書（`doc`）やWebサイト（`site`）だけでなく、**16:9 プレゼンテーションスライド**（`slide`）の作成をファーストクラスでサポートしています。

PowerPoint や Google Slides などの GUI ツールを使わずに、Markdown と Python 作図コードだけで、学会発表・カンファレンス・社内勉強会向けの美しいスライドデッキをコード管理できます。

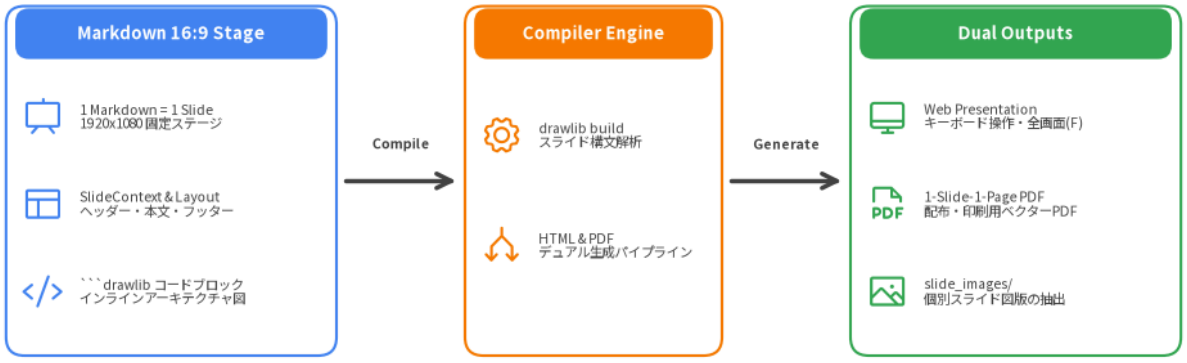


図 12.1: Drawlib スライド生成アーキテクチャとデュアル出力

12.1 スライドプロジェクトの作成 (`drawlib init slide`)

スライドプロジェクトは `drawlib init slide` コマンドで一発で足場生成できます:

```
# 標準のスライドプロジェクトを生成（slide_src/ が作成されます）
drawlib init slide

# プロジェクト名やテーマ、言語を指定する場合
drawlib init slide my_presentation -s google -l ja
```

生成されるディレクトリ構成

```
my_presentation/
├── slide_src/                # [正本] スライドの Markdown ソース
│   ├── 01_title.md           # タイトルスライド
│   ├── 02_agenda.md          # アジェンダ
│   ├── 03_architecture.md    # アーキテクチャ図解スライド
│   ├── styles.py             # スライド全体のスタイル・テーマ定義
│   ├── utils.py              # スライド専用のレイアウト関数
│   ├── slide.js              # プレゼンテーション再生・キーボード制御エンジン
│   ├── build.sh              # 全ターゲット一括ビルドスクリプト
│   ├── build_html.sh         # Web プレゼン生成スクリプト
│   ├── build_pdf.sh          # 1スライド1ページの PDF 生成スクリプト
│   └── serve.sh              # プレゼン用ローカルサーバー起動スクリプト
├── slide_html/               # [成果物] Web プレゼンテーション (index.html)
├── slide.pdf                 # [成果物] ベクター PDF プレゼンテーション
└── slide_images/             # [成果物] 抽出された高解像度図面画像
```

12.2 16:9 プレゼンテーションの執筆モデル

Drawlib のスライドは、**「1 つの Markdown ファイル ＝ 1 つのスライド」** という極めて直感的なモデルを採用しています。

- ファイル名の昇順（`01_title.md`, `02_agenda.md`, `03_overview.md` など）でスライドの順番が決定されます。
- 各スライドは `1920 × 1080` ピクセルの固定ステージにマッピングされ、プロジェクターやブラウザのウィンドウサイズに合わせて自動的にアスペクト比を維持しながらスケールします。

スライド Markdown の記述例

````
# マイクロサービスアーキテクチャの刷新

<div class="subtitle">イベント駆動型アーキテクチャへの段階的移行計画</div>

```drawlib width=1400px align=center file=arch_slide.png
from drawlib.canvas import setup
from drawlib.shapes import rectangle
from drawlib.lines import line
from drawlib.styles import Styles

setup(width=160, height=45)
rectangle((30, 22.5), width=35, height=18, style=Styles.Neutral, text="API Gateway")
rectangle((80, 22.5), width=35, height=18, style=Styles.PrimaryFlat, text="Event Broker",
text_style=Styles.WhiteBold)
rectangle((130, 22.5), width=35, height=18, style=Styles.SecondaryNeutral, text="Order Worker")
line((47.5, 22.5), (62.5, 22.5), arrow_head="->", style=Styles.DarkBold)
line((97.5, 22.5), (112.5, 22.5), arrow_head="->", style=Styles.DarkBold)
```
````

12.3 デュアル出力と発表者体験

スライドプロジェクトをビルドすると、利用シーンに応じた 2 つの形式が同時に生成されます:

1. **インタラクティブ Web プレゼンテーション (`slide_html/index.html`)**:
    - 会場での登壇やオンライン勉強会向けの HTML5 プレゼンテーション。
    - **キーボードナビゲーション**:
        - `Space` / `→` / `PageDown`: 次のスライドへ進む
        - `←` / `PageUp`: 前のスライドへ戻る
        - `F`: フルスクリーン（全画面）表示の切り替え
        - `O` / `Esc`: スライド全体をグリッド一覧表示するオーバービューモード
2. **1 スライド 1 ページのベクター PDF (`slide.pdf`)**:
    - 参加者への事前配布、DocSend / SpeakerDeck へのアップロード、印刷向けの PDF。
    - Playwright によるヘッドレスレンダリングにより、各スライドが 1 ページずつ欠けのない 16:9 ベクター PDF として忠実に焼き込まれます。